Express.js — Day 10

Route-Specific Middleware & Multiple Middleware on One Route

■ Day 10 Goal

Learn how to apply middleware only to selected routes, how to attach multiple middleware functions to one route, and why separating responsibilities makes middleware reusable and easier to maintain.

1. Global/Broad Middleware vs Route-Specific Middleware

A middleware registered with **app.use()** is registered broadly and can run for requests that reach that point in the application. Middleware placed directly in a route definition becomes part of that specific route's request-processing chain.

```
// Broad middleware
app.use(checkUser);

// Route-specific middleware
app.get("/dashboard", checkUser, (req, res) => {
    res.send("Dashboard");
});
```

2. Route-Specific Middleware

Example: a dashboard may require a user check, while the home page does not.

```
const checkUser = (req, res, next) => {
    console.log("Checking user...");
    next();
};

app.get("/dashboard", checkUser, (req, res) => {
    res.send("Dashboard");
});

app.get("/", (req, res) => {
    res.send("Home");
});
```

Requesting **GET /dashboard** runs **checkUser** and then the dashboard handler. Requesting **GET /** does not use **checkUser** because it was not attached to that route.

3. Multiple Middleware on One Route ■

Express allows multiple middleware functions to be listed before the final route handler.

```
const checkUser = (req, res, next) => {
    console.log("Checking user...");
    next();
};

const checkPermission = (req, res, next) => {
    console.log("Checking permissions...");
    next();
};
```

```
app.get("/dashboard", checkUser, checkPermission, (req, res) => {
    res.send("Dashboard");
});
```

The execution flow is:

```
GET /dashboard
      ↓
checkUser
      ↓ next()
checkPermission
      ↓ next()
route handler
      ↓
Dashboard
```

4. Why next() Is Critical

Each middleware must either finish the request by sending a response or call **next()** to pass control forward. If **checkUser** does not call **next()** and does not send a response, the request can hang. The next middleware and route handler will not execute.

5. Separation of Responsibility

It is better to keep meaningful responsibilities in separate middleware functions rather than putting everything into one large middleware. For example, authentication and permission checking can be separate concerns.

```
const checkUser = (req, res, next) => {
    // authentication-related logic
    next();
};

const checkPermission = (req, res, next) => {
    // permission-related logic
    next();
};
```

Separate middleware can then be reused in different combinations:

```
/dashboard → checkUser → checkPermission
/admin     → checkUser → checkPermission
/profile   → checkUser
```

Memory rule: One middleware should have one meaningful responsibility and remain reusable where possible.

6. Day 10 Practical Experiment

You tested **/dashboard** with route-specific middleware and then added a second middleware.

```
app.get("/dashboard", checkUser, checkPermission, (req, res) => {
    res.send("Dashboard");
});
```

Your terminal showed:

```
Checking user...
Checking permissions...
```

This confirmed that both middleware functions executed in the registered order before the dashboard route handler.

7. Real-World Mental Model

A protected route can be thought of as a chain of checks before the final controller/handler:

```
Request
  ↓
authenticateUser
  ↓
checkPermission
  ↓
validateRequest
  ↓
controller
  ↓
Response
```

This pattern will become especially important later when learning Authentication + JWT and Authorization.

8. Day 10 Knowledge Check — Answers

1. Difference between app.use(checkUser) and app.get('/dashboard', checkUser, handler)?

app.use(checkUser) registers the middleware broadly for requests reaching that point. Putting checkUser inside the route makes it part of that specific route's middleware chain.

2. What if checkUser does not call next()?

The request can hang, and control will not move to checkPermission or the route handler unless checkUser sends a response.

3. Why separate checkUser and checkPermission?

Separate middleware keeps responsibilities distinct and makes each piece reusable across the routes that need it.

9. Long-Term Memory Summary

• **app.use()** registers middleware broadly.

• Middleware placed in a route applies to that route's request chain.

• A route can have multiple middleware functions.

• Middleware executes in the order it is listed.

• Each middleware should call **next()** to continue unless it finishes the response.

• Separate middleware improves responsibility separation and reusability.

• Authentication → permission → validation → handler is a common middleware-chain mental model.

■ Day 10 Complete

Express.js progress: 10 / 15 days → 66.7%

Day 10 is officially complete. You now understand both broad middleware registration and route-specific middleware chains.